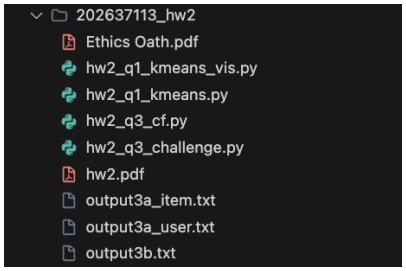
Introduction to Data Mining, Spring 2026
HW2

Due date: 30/04/2026 (11:59pm)

Submission instructions: Use SNU myETL to submit your homeworks. Your submission should be one gzipped tar file whose name is `YourStudentID_hw2.tar.gz`. For example, if your student ID is 2023-11111, and it is for homework #2, please name the file as `202311111_hw2.tar.gz`. You can also use these extensions: `tar, gz, zip, tar.zip`.

Your zip file should include the following contents: one PDF file containing your written answers (`hw2.pdf`); four Python source codes (`hw2_q1_kmeans.py`, `hw2_q1_kmeans_vis.py`, `hw2_q3_cf.py`, and `hw2_q3_challenge.py`); three text files with the outputs (`output3a_user.txt`, `output3a_item.txt`, `output3b.txt`), and the Ethics Oath pdf file. Before zipping your files, please make a directory named `YourStudentID_hw2` and put your files in the directory. Then, compress the directory to make a zipped file.



If you do not follow this format, we will **deduct** 1 point from your total score.

Submitting writeup: Prepare answers to the homework questions into a single PDF file (`hw2.pdf`). You can use the following template. Please write as succinctly as possible. For questions that require Python programming, **please attach your code snippets to your document** at the corresponding question section.

Submitting code: Each problem is accompanied by a programming part. Put all the code for each question into a single file. Please make sure your code is well structured and has descriptive comments. We will **deduct** 1 point per line if your attached code snippets is different from your actual program.

Ethics Oath: For every homework submission, please fill out and submit the **PDF** version of this document that pledges your honor that you did not violate any ethics rules required by this course and SNU. You can either scan a printed version into a PDF file or make the Word document into a PDF file after filling it out. Please sign on the document

and submit it along with your other files.

Use of AI Tools: The use of AI tools for programming is not recommended in this course. Nevertheless, if you use any AI tool in completing this homework, you must disclose it in `hw2.pdf`. At the beginning of your PDF, briefly describe which AI tool(s) you used, how you used them, and what assistance they provided. There will be no point deduction for the use of AI tools, as long as the disclosure is complete and truthful. If you did not use any AI tool, you may optionally state that no AI tools were used.

Discussions with other people are permitted and encouraged. However, when the time comes to write your solution, such discussions (except with course staff members) are no longer appropriate: you must write down your own solutions independently. If you received any help, you must specify on the top of your written homework any individuals from whom you received help, and the nature of the help that you received. *Do not, under any circumstances, copy another person's solution.* We check all submissions for plagiarism and take any violations seriously.

[**Note**] There are answers to frequently asked questions at the end of this document. Please refer to it.

1 Clustering (30 points)

1.1 [12 pts] Mathematical Understanding of k-Means Clustering

(a) [6 pts] **Variance Decomposition of the k-Means Objective**

The k-means clustering algorithm partitions a dataset into k clusters by minimizing the *within-cluster sum of squared distances*. Let $X = \{x_1, x_2, \ldots, x_n\}$ be a set of data points in $\mathbb{R}^d$, and suppose they are partitioned into k clusters $C_1, C_2, \ldots, C_k$.

Let

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

denote the *global mean* of the dataset, and let

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

denote the centroid of cluster C_j.

Define the following quantities:

- Total Sum of Squares (TSS): $\sum_{i=1}^{n} \|x_i - \mu\|^2$
- Within-Cluster Sum of Squares (WCSS): $\sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$
- Between-Cluster Sum of Squares (BCSS): $\sum_{j=1}^{k} |C_j| \, \|\mu_j - \mu\|^2$

Problem. Show that the total variance of the dataset can be decomposed as

$$\text{TSS} = \text{WCSS} + \text{BCSS}.$$

Based on this result, explain why minimizing the k-means objective (WCSS) is equivalent to maximizing the between-cluster variance (BCSS).

(b) [6 pts] **Matrix Formulation of the k-Means Objective**

The k-means clustering can be interpreted as a matrix approximation problem.

Let $X \in \mathbb{R}^{n \times d}$ denote the data matrix whose i-th row corresponds to the data point $x_i^\top$. Suppose the data are partitioned into k clusters $C_1, C_2, \ldots, C_k$.

Define the cluster indicator matrix $H \in \mathbb{R}^{n \times k}$ as

$$H_{ij} = \begin{cases} \frac{1}{\sqrt{|C_j|}}, & \text{if } x_i \in C_j \\ 0, & \text{otherwise.} \end{cases}$$

It can be verified that $H^\top H = I_k$.

Problem.

(i) [3 pts] Show that the i-th row of $HH^\top X$ corresponds to the centroid of the cluster to which x_i belongs.

(ii) [3 pts] Using this result, show that the k-means objective function

$$\sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

can be written equivalently as

$$\|X - HH^\top X\|_F^2,$$

where $\|\cdot\|_F$ denotes the Frobenius norm.

1.2 [18 pts] Distributed k-Means with Spark

In this problem, you will implement the classic k-means clustering algorithm using **Apache Spark**. The goal is to cluster a large collection of data points and analyze the behavior of the clustering objective under different numbers of clusters.

Your implementations must be written in two Python programs named `hw2_q1_kmeans.py` and `hw2_q1_kmeans_vis.py`. The dataset file provided via myETL (`kmeans_data.txt`) contains one data point per line in the following format `<feature_1> <feature_2> ... <feature_d>` where each feature is a floating-point value.

(a) [10 pts] **Implementation**

Implement the distributed k-means algorithm using Spark RDD operations and the visualization of the result. With the given skeleton codes `hw2_q1_kmeans.py` and `hw2_q1_kmeans_vis.py`, look for `##TODO##` sections and fill them in.

(b) [5 pts] **Choosing the number of clusters**

Run your implementation with various k's following the command below:

```
spark-submit hw2_q1_kmeans.py path/to/kmeans_data.txt \
2 15 output --test-and-plot
```

Then report the plot `elbow_plot.png`. Based on the resulting plot:

(i) [1 pt] Identify the approximate *elbow point*.

(ii) [2 pt] Explain why WCSS monotonically decreases as k increases.

(iii) [2 pts] Discuss why choosing a very large k may not be desirable in practice.

(c) [3 pts] **Visualization and cluster interpretation**

Re-run the clustering with the found proper k from the elbow point (replace the argument k to the actual number of clusters for the command below):

```
spark-submit hw2_q1_kmeans.py path/to/kmeans_data.txt k 15 output
```

(i) [1 pt] Run the code with the following command and report the resulting plot:

```
python hw2_q1_kmeans_vis.py path/to/kmeans_data.txt \
path/to/centroids.txt cluster_plot.png
```

(ii) [2 pts] Briefly describe the characteristics of the resulting clusters.

2 Dimensionality Reduction (20 points)

2.1 [20 pts] Mathematical Understanding of Power Iteration and PCA

(a) [10 pts] **Successive Eigenvector Extraction**

Let $S \in \mathbb{R}^{d\times d}$ be a real symmetric matrix with eigenvalues $\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_d$ and corresponding orthonormal eigenvectors $\{v_1, \ldots, v_d\}$.

Define the deflated matrix

$$S' = S - \lambda_1 v_1 v_1^\top.$$

(i) [2 pt] Show that v_1 lies in the null space of S'.

(ii) [2 pts] Show that for all $i \geq 2$, $S' v_i = \lambda_i v_i$.

(iii) [3 pts] Let $t \in \mathbb{R}^d$ with $\|t\|_2 = 1$. Prove that

$$\max_{\substack{\|t\|_2=1 \\ t\perp v_1}} t^\top S t = \max_{\|t\|_2=1} t^\top S' t.$$

(iv) [3 pts] Using the result above, explain why applying power iteration to S' recovers the second largest eigenvector v_2.

(b) [10 pts] **Effect of Centering on Principal Directions**

Let $M \in \mathbb{R}^{n\times d}$ be a data matrix, and let

$$\mu = \frac{1}{n}\sum_{i=1}^{n} M_{i,:} \in \mathbb{R}^d$$

be the mean vector of the rows.

Define the centered matrix $\widetilde{M} = M - \mathbf{1}\mu^\top$, where $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector.

Let $S = M^\top M$ and $\widetilde{S} = \widetilde{M}^\top \widetilde{M}$.

(i) [2 pts] Show that $S = \widetilde{S} + n\,\mu\mu^\top$.

(ii) [3 pts] Let $t \in \mathbb{R}^d$ and $\|t\|_2 = 1$. Show that

$$t^\top S t = t^\top \widetilde{S} t + n(t^\top \mu)^2.$$

Using this expression, explain the meaning of each term $t^\top \widetilde{S} t$ and $n(t^\top \mu)^2$.

(iii) [5 pts] Assume that $\|\mu\|_2^2$ is sufficiently large compared to the spectral norm of $\widetilde{S}$. Argue that the leading eigenvector of S is dominated by the mean direction μ, whereas the leading eigenvector of $\widetilde{S}$ captures the direction of maximum centered variance. Conclude why centering matters in PCA.
(hint: let $u = \mu/\|\mu\|_2$ and assume any unit vector t s.t. $t \perp \mu$)

3 Recommendation Systems (50 points)

3.1 [20 pts] Implement collaborative filtering

In this problem, you will implement two classical neighborhood-based collaborative filtering methods: *user-based collaborative filtering* and *item-based collaborative filtering* as described in MMDS Chapter 9. You will run both methods on a real movie-rating dataset.

The dataset files are provided on myETL. The file `ratings.txt` contains rating records and each line has the format: `<USER ID>,<MOVIE ID>,<RATING>,<TIMESTAMP>` where `<USER ID>` is the user ID, `<MOVIE ID>` is the movie ID, `<RATING>` is the rating given by the user to the movie, taking values in $\{0.5, 1.0, 1.5, \ldots, 5.0\}$, `<TIMESTAMP>` is the timestamp of the rating. The file `movies.txt` contains movie metadata. It may be useful for inspecting the dataset.

Let $r_{u,i}$ denote the original rating that user u gave to movie i, if it exists.

For computing similarities, use the *normalized* rating matrix obtained by mean-centering each user's ratings. You must use these normalized ratings *only for computing similarity.* After similar users or similar items are found, prediction must be performed using the *original* ratings, together with the weighted residual formula described below.

Similarity between two users or between two items must be computed only on their *common observed ratings.*

- For two users u and v, let I_{uv} be the set of movies rated by both users. Their similarity is computed only over I_{uv}.
- For two items i and j, let U_{ij} be the set of users who rated both items. Their similarity is computed only over U_{ij}.

In addition, impose a minimum overlap condition: if the number of common ratings is smaller than a fixed threshold m, then the similarity is treated as invalid and that user/user or item/item pair must not be used as neighbors. In this problem, use $m = 5$.

For *user-based collaborative filtering*, perform the following steps.

1. Find all users $v \neq u$ such that the user v has rated movie i, and the similarity between u and v is valid under the minimum-overlap condition.
2. Among them, choose the top-10 users with the highest cosine similarity to u. If fewer than 10 valid neighbors exist, use all of them.
3. Let the selected neighbor set be $N_u(i)$. Predict the rating by the similarity-weighted residual formula
$$\hat{r}_{u,i}^{\text{user}} = \bar{r}_u + \frac{\sum_{v \in N_u(i)} s(u,v)\big(r_{v,i} - \bar{r}_v\big)}{\sum_{v \in N_u(i)} |s(u,v)|},$$
where $s(u,v)$ is the cosine similarity between users u and v.
4. Finally, clip the predicted value into the valid rating range $[0.5, 5.0]$.

For *item-based collaborative filtering*, perform the following steps.

1. Find all movies $j \neq i$ such that the user u has rated movie j, and the similarity between movies i and j is valid under the minimum-overlap condition.
2. Among them, choose the top-10 movies with the highest cosine similarity to movie i. If fewer than 10 valid neighbors exist, use all of them.
3. Let the selected neighbor set be $N_i(u)$. Predict the rating by the similarity-weighted residual formula
$$\hat{r}_{u,i}^{\text{item}} = \bar{r}_i + \frac{\sum_{j \in N_i(u)} s(i,j)\big(r_{u,j} - \bar{r}_j\big)}{\sum_{j \in N_i(u)} |s(i,j)|}$$
where $s(i,j)$ is the cosine similarity between items i and j.
4. Finally, clip the predicted value into the valid rating range $[0.5, 5.0]$.

If the denominator in the prediction formula is zero, use the fallback prediction $\hat{r}_{u,i} = \bar{r}_u$ for user-based CF and $\hat{r}_{u,i} = \bar{r}_i$ for item-based CF.

Prediction task. Consider the target user u whose ID is `414` in `ratings.txt`. For this user, we assume that all ratings on movies with IDs from 1 to 1000 are hidden. Your task is to predict the user's ratings on these candidate movies. Run the code with the following command; `python hw2_q3_cf.py path/to/ratings.txt` and it must generate the following two output files.

1. `output3a_user.txt`

   Using the *user-based* method, predict the ratings of user $u = 414$ for all candidate movies with IDs 1 to 1000. Among these movies, select the top-5 movies with the highest predicted ratings. Write them in descending order of predicted rating, one line per movie, in the format

   ```
   <MOVIE ID><TAB><PREDICTED RATING>
   ```

2. `output3a_item.txt`

   Using the *item-based* method, predict the ratings of user $u = 414$ for all candidate movies with IDs 1 to 1000. When computing item-item similarity for a target movie i, you may use all other movies rated by user u as possible neighbors, except the candidate movie i itself. Among the candidate movies with IDs 1 to 1000, select the top-5 movies with the highest predicted ratings. Write them in descending order of predicted rating, one line per movie, in the format

   ```
   <MOVIE ID><TAB><PREDICTED RATING>
   ```

Tie-breaking rule. If multiple movies have exactly the same predicted ratings, break ties by sorting movie IDs in ascending order and choose the smaller movie ID first.

Please submit the following results:

- Your code snippets (in `hw2.pdf`)
- Your source implementation (`hw2_q3_cf.py`)
- Output text files (`output3a_item.txt`, `output3a_user.txt`)

3.2 [30 pts] Movie Recommendation Challenge

Similar to the NetFlix Challenge, we will have an IDM26 Movie Recommendation Challenge. While part 3.1 focuses on an explicit setting where predicting the ratings directly, this part deals with an implicit setting—assuming that rated movies correspond to a user preference (1), and unrated movies correspond to non-preference (0).

Your task is to extend and enhance the recommender system developed in part 3.1 using any technique of your choice—such as improved normalization, clustering, alternative similarity measures, matrix factorization (UV-decomposition), or even advanced methods inspired by the Netflix Prize winners. The goal is to predict whether a movie is likely to be a user's preference.

Use the same training dataset as in the previous part. The test dataset (without labels, which you are required to fill in for submission) for this part can be found at myETL.

`ratings_test.txt`

This file consists of 20,000 lines where each line is in the following format:

```
<USER ID>,<MOVIE ID>,<TIMESTAMP>
```

The 10,000 rows represent movies `<MOVIE ID>` that the user with the corresponding `<USER ID>` has actually rated, while the remaining rows represent movies that the user has not rated. Please fill in the `<SCORE FOR MOVIE>` column (scores should be real numbers) so that each line in the output file is the following format:

```
<USER ID>,<MOVIE ID>,<SCORE FOR MOVIE>,<TIMESTAMP>
```

For this problem, the exact prediction of the rating for movie is not our primary focus. In recommender system domains, it is generally assumed that users prefer items they have interacted with over those they have not. Based on this assumption, we will evaluate how well your model captures user preferences by analyzing the `<SCORE FOR MOVIE>` returned by your own recommender system.

We will compute the ROC-AUC score for your recommender system and award points accordingly. The higher ROC-AUC score, the more points you will receive. Even simple or trivial solutions will receive some points, but higher-performing systems will be awarded more.

The ROC (Receiver Operating Characteristic) curve is a graphical representation of a classifier's performance across all threshold values. It plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR)** at different thresholds.
The AUC (Area Under the Curve) provides a single scalar value representing the overall performance of the classifier. The AUC ranges from 0 to 1:

- **AUC = 1**: Perfect classifier
- **AUC = 0.5**: Random guessing

- **AUC < 0.5**: Classifier performs worse than random guessing

The performance of your model will be evaluated based on the predictions it makes for each interaction in the test dataset.

Each interaction produces result, leading to a total of 20,000 classification results for the test sets. If you want to check your model's ROC-AUC score, **make validation data** from `ratings.txt` and check performance. We give you pseudo code which will be used for evaluate your model which is uploaded at myETL: `ROC_AUC_score.py`

After run, your code should **save an** `output3b.txt` **file in the current directory** automatically. An example of output file generation is provided in `hw2_q3_challenge.py`. You may modify the code as needed, but ensure that your output file follows the exact same format.

Please submit the following results:

- A detailed description of the techniques used. (`hw2.pdf`)
- Your source code implementation (`hw2_q3_challenge.py`).
- Filled out test file (`output3b.txt`).

4 Answers to Frequently Asked Questions

- Please write your answers for hand-writing questions as sufficiently as possible. If your answers are somewhat vague or directly concludes to the final result without explaining the process, scores may be deducted.
- You can only use pre-imported libraries on the skeleton codes. If you use any other libraries, your code implementation points will be 0. The libraries are already installed under `IDM26` conda environment.
- Each `##TODO##` blocks will be graded proportionally to the total points of implementation part.
- Your codes will be evaluated under same environment on the DM01 server using TA's labtop.
- **Questions about Problem 1**
  - If your code implementations fail to run using the following commands on TA's labtop, your implementation points will be 0 (with `k` replaced to proper number of clusters).
    * `spark-submit hw2_q1_kmeans.py path/to/kmeans_data.txt k 15 output`
    * `python hw2_q1_kmeans_vis.py path/to/kmeans_data.txt \ path/to/centroids.txt cluster_plot.png`
  - For `hw2_q1_kmeans.py`, your code should run in 5 minutes on TA's labtop. For `hw2_q1_kmeans_vis.py`, your code should run in 10 seconds on TA's labtop. If any of your code fails to run in time, your implementation score will be 0.
- **Questions about Problem 3.1**
  - If your code implementation fails to run using the following command on TA's labtop, your implementation points will be 0.
    * `python hw2_q3_cf.py path/to/ratings.txt`
  - Your code should run in 1 minute on TA's labtop. If it fails to run in time, your implementation score will be 0.
  - You should still attached your code snippets on `hw2.pdf` in addition to submitting `hw2_q3_cf.py` and the resulting files `output3a_user.txt` and `output3a_item.txt`.
- **Questions about Problem 3.2**
  - Your code must run from the command line as follows; `python hw2_q3_challenge.py path/to/ratings.txt path/to/ratings_test.txt`
  - There is no time limit for running this code.
  - You are allowed to use `pandas` and `scikit-learn` libraries only for this problem.